FCFS/SJF algorithm code
scheduling.js

```
// Non-preemptive CPU scheduling algorithms: FCFS and SJF.
// Input: array of { id, arrival, burst } (arrival/burst are non-negative numbers).
// Output: schedule (for the gantt chart), per-process results, and averages.


function buildResult(order, processesById) {
 let clock = 0
 const schedule = []
 const rows = []

 for (const id of order) {
  const p = processesById[id]
  const start = Math.max(clock, p.arrival)
  const finish = start + p.burst
  clock = finish

  const turnaround = finish - p.arrival
  const waiting = turnaround - p.burst

  schedule.push({ id: p.id, start, finish, burst: p.burst })
  rows.push({
   id: p.id,
   arrival: p.arrival,
   burst: p.burst,
   start,
   finish,
   turnaround,
```

FCFS/SJF algorithm code
scheduling.js

```
    waiting,
  })
 }

 const totalWaiting = rows.reduce((sum, r) => sum + r.waiting, 0)
 const totalTurnaround = rows.reduce((sum, r) => sum + r.turnaround, 0)
 const n = rows.length || 1

 return {
  schedule,
  rows,
  averageWaiting: totalWaiting / n,
  averageTurnaround: totalTurnaround / n,
 }
}

export function runFCFS(processes) {
 const processesById = Object.fromEntries(processes.map((p) => [p.id, p]))
 // FCFS: order of arrival; ties broken by original submission order.
 const order = [...processes]
  .sort((a, b) => a.arrival - b.arrival || a.__index - b.__index)
  .map((p) => p.id)
 return buildResult(order, processesById)
}

export function runSJF(processes) {
```

FCFS/SJF algorithm code
scheduling.js

```
 const processesById = Object.fromEntries(processes.map((p) => [p.id, p]))

 const remaining = [...processes]

 const order = []

 let clock = 0


 while (remaining.length > 0) {

  const available = remaining.filter((p) => p.arrival <= clock)


  let next

  if (available.length > 0) {

   // Shortest burst among arrived processes; ties -> earliest arrival,

   // then original submission order (FCFS tiebreak, per spec).

   next = available.sort(

    (a, b) =>

     a.burst - b.burst ||

     a.arrival - b.arrival ||

     a.__index - b.__index

   )[0]

  } else {

   // CPU idle until the next process arrives.

   next = remaining.sort((a, b) => a.arrival - b.arrival)[0]

   clock = next.arrival

  }


  order.push(next.id)

  clock += next.burst
```

FCFS/SJF algorithm code
scheduling.js

```
    remaining.splice(remaining.indexOf(next), 1)
  }


  return buildResult(order, processesById)
}


export function runSchedule(algorithm, processes) {
  const withIndex = processes.map((p, i) => ({ ...p, __index: i }))
  return algorithm === "sjf" ? runSJF(withIndex) : runFCFS(withIndex)
}
```